



Laboratorio Interdisciplinare A

Manuale Tecnico: TheKnife
a.a 2024/2025

Data: 07/06/2025

Autore: Luca Cavagnari, 761291



TheKnife — Indice

1. *Arc Arc*

2. *Strutture Dati*

3. *Algoritmi Utilizzati*

4. *Limiti della Soluzione*

5. *Sitografia / Bibliografia*

6. *JavaDoc*

Il progetto è suddiviso in pacchetti secondo il principio della separazione delle responsabilità: ogni componente ha un proprio dominio ben definito. Le entità, la logica di caricamento, le costanti e l'interfaccia utente sono tutte collocate in moduli specifici, per facilitare riuso e manutenibilità. Questo per facilitare l'estensione futura, grazie ad una separazione netta tra le componenti, in particolare:

- Lettura e salvataggio dei dati persistenti da file,
- Gestione della logica relativa a utenti e ristoranti (e relativi attributi),
- Interazione con l'utente (input/output) tramite menu testuali,
- Sanificazione degli input e adeguati controlli, per prevenire errori comuni.

Le classi sono organizzate in una struttura gerarchica, divisa in due macro-entità, con entità "minori" al loro "servizio", come ad esempio gli enumerativi. Esse sono:

- JsonEntity: superclasse astratta, comune a tutte le entità persistenti. Contiene l'identificativo univoco (UUID), il file di salvataggio associato, convenzionalmente chiamato "<uuid>.json", e la logica di serializzazione in formato JSON.
- User: superclasse astratta per la gestione della logica degli utenti. Sono previsti due sottotipi: Client (client) e Owner (gestore o proprietario di un ristorante). Questo permette di aggiungere altri tipi di utenti senza duplicare il codice e facilitare il debugging

Altre classi hanno collegamenti gerarchici ad altre entità, anche se non definiti nella struttura di "supertipo/sottotipo". Questi, sono generalmente associazioni per creare un modello di relazione 1 a 1 oppure tanti a 1 tra 2 entità.

Ad esempio, la lista "Ristoranti preferiti" associata con il profilo di un cliente, mettendo così in relazione uno o più ristoranti con un singolo cliente (e viceversa tanti clienti con 1 ristorante); oppure, mettendo in relazione una recensione con l'utente che l'ha scritta, dandogli la possibilità di accedervi e gestirla a proprio piacimento.

Il sistema fa anche uso di dipendenza esterne tramite il manager Maven per ottimizzare le prestazioni e semplificare la scrittura di codice. Esse sono:

- Lombok, libreria che automatizza Getter e Setter, con altri comuni metodi, attraverso notations, permettendo di avere codice pulito e leggibile, mantenendo la modularità, con pieno controllo di quali campi sono accessibili.
-
- Jackson, libreria per la gestione di serializzazione in file JSON, con cui, tramite metodi Builder permette di creare oggetti ed Array JSON modularmente e con semplicità.
-
- Libphonenumber, Libreria che permette di ottenere il prefisso nazionale telefonico, sviluppata da Google.

Il sistema di persistenza dei dati è progettato per essere semplice, ma efficace: ogni entità viene salvata nel proprio file JSON all'interno di una cartella dedicata in base al tipo:

- users/ contiene i dati degli utenti,
- restaurants/ contiene i dati dei ristoranti e delle loro recensioni,

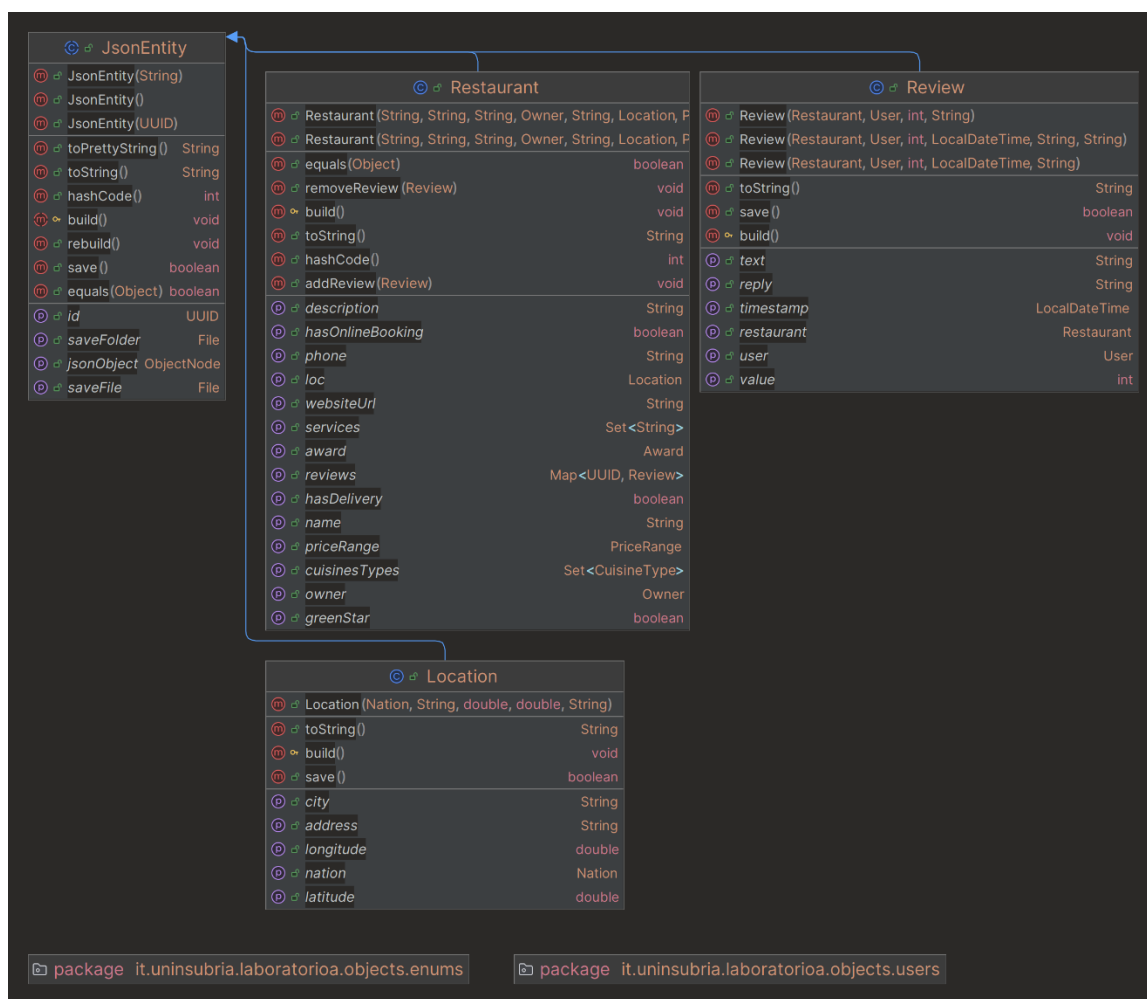
Il sistema applica il Template Method per la costruzione degli oggetti e il loro salvataggio su file, la responsabilità è quindi delegata tra entità correlate per mantenere coerenza dei dati.

Le recensioni, invece, sono annidate nel file del ristorante, a cui sono riferite. Questo design consente operazioni di lettura e scrittura mirate e riduce la complessità di sincronizzazione. Viene adottato un approccio "File-Based" evitando la necessità di database esterni e connessioni.

Un potenziale problema che potrebbe minare le prestazioni di caricamento dei dati all'avvio del programma, è l'accumulo di recensioni a lungo termine nello stesso file. Una soluzione al problema è separare le recensioni dai ristoranti, in termini di memoria permanente, delegando la loro gestione alla classe JsonEntity come entità a se stante, anziché come attributo di Restaurant.

A livello di pacchetti:

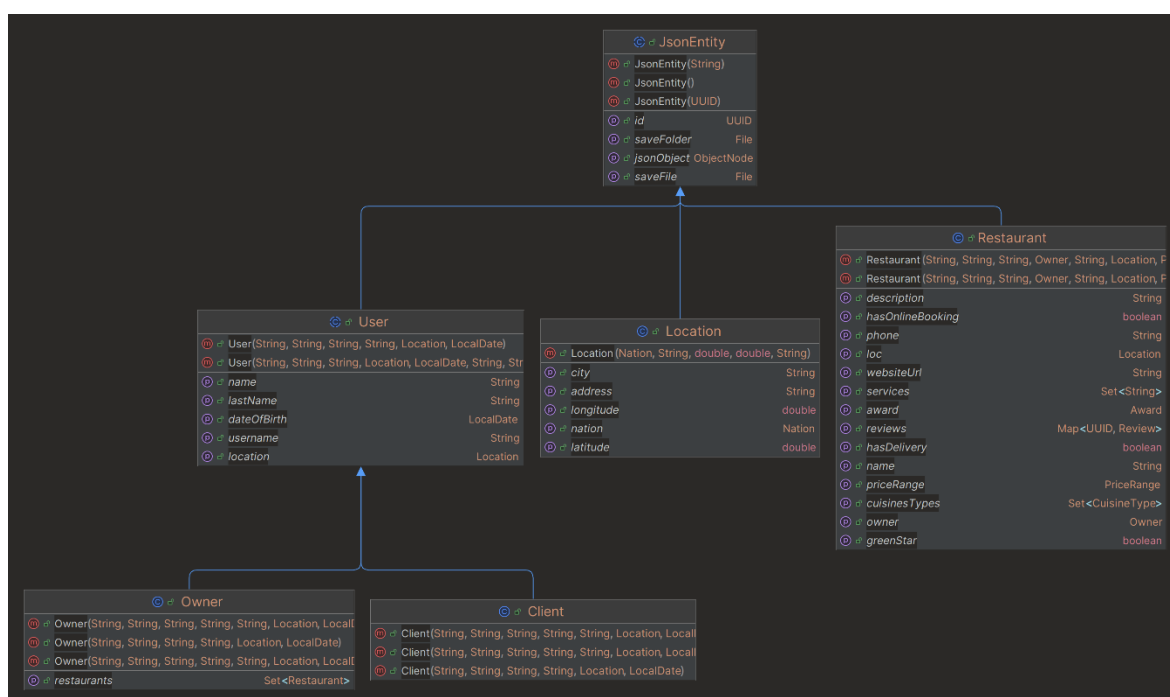
- **it.uninsubria.laboratorioa.objects:** contiene le principali classi di dominio, fondamentali per il sistema, quali JsonEntity, Restaurant, Person, Review e Location. Tutte queste classi estendono la superclasse astratta JsonEntity, la quale implementa il pattern “*Template Method*”, per la gestione della serializzazione e per la persistenza dei dati in formato JSON. Ciò consentirebbe di centralizzare la logica di salvataggio e di rigenerazione degli oggetti da dati persistenti.



- **it.uninsubria.laboratorioa.users:** Contiene le classi di dominio responsabili per la gestione degli utenti, divisi nei loro 2 tipi, ognuno con le proprie funzionalità e limitazioni.

La superclasse User, anch'essa sottotipo di un'altra classe, JsonEntity, gestisce i dati comuni (nome, cognome, data di nascita, username, password,...), oltre che a effettuare operazioni di hash sulla password, al momento della registrazione o del cambio password, sanifica prontamente i dati alla sorgente attraverso, per esempio, intervalli di lunghezza accettabili (username, nome e cognome), limitando gli errori più comuni e le incostistenze nei dati.

Le password vengono protette tramite l'algoritmo di Hash PBKDF2, utilizzando un salt, casuale, per ogni utente, di 16bit, con una chiave di 256bit e 1000 iterazioni dell'algoritmo per garantire la sicurezza. Salt ed hash vengono serializzati insieme ai dati dell'utente per garantire il corretto funzionamento del sistema.



- **it.uninsubria.laboratorioa.enums:** racchiude le classi enumerative per rappresentare in modo sicuro e controllato i valori fissi come nomi di Nazioni (Nation) ed i dati di Michelin, come: fasce di prezzo per i ristoranti (PriceRange) e il numero di stelle ottenute da un dato ristorante. Questo garantisce consistenza semantica e facilità di manutenzione, aggiungendo però la necessità di sanificare gli input e limitare le incosistenze.

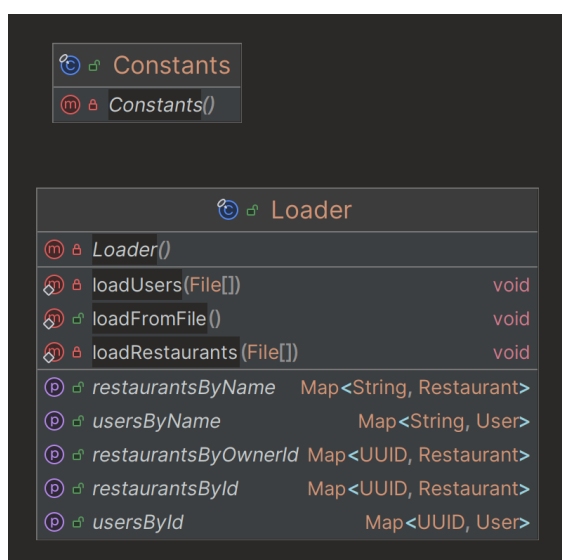
E ↗ CuisineType	
Ⓜ ↗ CuisineType ()	
↗ values()	CuisineType []
Ⓜ ↗ toString()	String
↗ valueOf(String)	CuisineType

E ↗ Award	
Ⓜ ↗ Award(int)	
↗ valueOf(String)	Award
↗ fromInt(int)	Award
Ⓜ ↗ toString()	String
↗ values()	Award []

E ↗ Nation	
Ⓜ ↗ Nation(String)	
↗ values()	Nation []
Ⓜ ↗ toString()	String
↗ valueOf(String)	Nation
Ⓟ ↗ isoCode	String

E ↗ PriceRange	
Ⓜ ↗ PriceRange (String, int, int)	
↗ valueOf(String)	PriceRange
↗ byDollarAmount(int)	PriceRange
↗ values()	PriceRange []
Ⓟ ↗ symbol	String
Ⓟ ↗ maxPrice	int
Ⓟ ↗ minPrice	int

- **it.uninsubria.laboratorioa.utils:** contiene classi di utilità, come ad esempio, costanti di progetto, inclusi formati per timestamp, configurazioni di serializzazione e parametri globali, e chiama la classe Loader. Essa, implementa la logica per leggere la memoria persistente, carica file JSON, e istanzia gli oggetti di dominio corrispondenti, rendendoli disponibili all'uso.



- **it.uninsubria.laboratorioa.ui:** Contiene classi per l'interfaccia testuale, responsabili della presentazione dei dati e della gestione dell'interazione utente. Include menù dinamici e metodi di input validati tramite la classe IO, riducendo al minimo il rischio di input malformati.



TheKnife – Strutture Dati

Questa sezione descrive le principali classi, formalmente note come entità, che compongono la struttura dati del Progetto e come esse interagiscono fra di loro. Ogni entità rappresenta un punto chiave del sistema ed è progettata per essere semplice, modulare e facilmente serializzabile su file JSON.

Ogni entità è un sottotipo di `JsonEntity`, una superclasse astratta che fornisce il necessario per creare un oggetto serializzabile e salvarlo su file.

Alcune validazioni, per evitare incoerenza nei dati, sono effettuate nelle singole classi, dato che sono effettuate su attributi specifici.

Segue una panoramica delle entità principali con attributi e logica rilevante

JsonEntity

Ruolo: superclasse astratta per tutte le entità strutturali, serializzazione di entità in JSON

Attributi principali: UUID id: identificatore univoco di oggetto

Relazioni: estesa da User, Restaurant, Review, Locaion

Validazioni: Il metodo save() controlla l'esistenza della cartella di destinazione (saveFolder) e la crea se assente

Note:

Implementa il pattern "Template Method" con override del metodo build() nelle sottoclassi.



JsonEntity	
JsonEntity(String)	
JsonEntity()	
JsonEntity(UUID)	
toPrettyString()	String
toString()	String
hashCode()	int
build()	void
rebuild()	void
save()	boolean
equals(Object)	boolean
id	UUID
saveFolder	File
jsonObject	ObjectNode
saveFile	File

Restaurant

Ruolo: Classe dominio che rappresenta un ristorante da promuovere.

Attributi principali:

- name(nome ristorante),
- description,
- websiteUrl (pagina web),
- phone (recapito telefonico con prefisso),
- location(posizione geografica),
- owner(utente proprietario o gestore),
- cuisinesTypes(stili di cucina offerti),
- reviews(lista di recensioni),
- services(lista di servizi offerti dal ristorante),
- priceRange(Range di prezzo secondo Michelin),
- award(Numero di stelle Michelin ottenute),
- greenStar(flag booleana per indicare se il ristorante ha ricevuto la “Green star” di Michelin),
- hasDelivery (flag per indicare se il ristorante offre la consegna a domicilio)
- hasOnlineBooking (flag per indicare se il ristorante permette la prenotazione online)

Restaurant	
Restaurant (String, String, String, Owner, String, Location, P	
Restaurant (String, String, String, Owner, String, Location, P	
equals(Object)	boolean
removeReview (Review)	void
build()	void
toString()	String
hashCode()	int
addReview (Review)	void
description	String
hasOnlineBooking	boolean
phone	String
loc	Location
websiteUrl	String
services	Set<String>
award	Award
reviews	Map<UUID, Review>
hasDelivery	boolean
name	String
priceRange	PriceRange
cuisinesTypes	Set<CuisineType>
owner	Owner
greenStar	boolean

Relazioni: Estende jsonEntity, collegato a Owner (proprietario ristorante), contiene Review e Location; aggrega tipi CuisineType, PriceRange, Award

Validazioni: valori null o vuoti gestiti con default sicuri; recensioni duplicate evitate in addReview()

Serializzazione: Estende jsonEntity, salva su file “restaurants/<id>.json”

Note: Utilizza pattern di “Serialization Dispatch” (via Jackson), Delegation (persistenza Review → Restaurant), Defensive Defaults

Review

Ruolo: superclasse astratta per tutte le entità strutturali, serializzazione di entità in JSON

Attributi principali: UUID id: identificatore univoco di oggetto

Relazioni: estesa da User, Restaurant, Review, Locaion

Validazioni: Il metodo save() controlla l'esistenza della cartella di destinazione (saveFolder) e la crea se assente

Note:

Implementa il pattern “Template Method” con override del metodo build() nelle sottoclassi.

Review	
Review (Restaurant, User, int, String)	
Review (Restaurant, User, int, LocalDateTime, String, String)	
Review (Restaurant, User, int, LocalDateTime, String)	
toString()	String
save()	boolean
build()	void
text	String
reply	String
timestamp	LocalDateTime
restaurant	Restaurant
user	User
value	int

User

Ruolo: superclasse astratta per la rappresentazione degli utenti del sistema (clienti e proprietari). Estende JsonEntity per abilitare persistenza e serializzazione.

Attributi principali: username (nome utente), name, lastName(cognome), location(posizione geografica), dateOfBirth (data di nascita), classe Hasher (salt + hash password)

Relazioni: JsonEntity; contiene Location; specializzata da Client e Owner

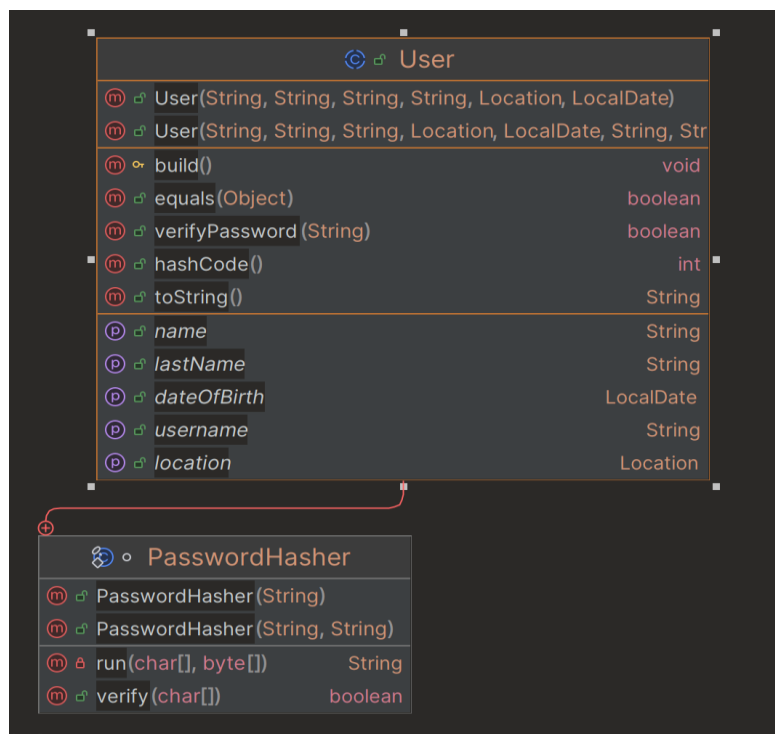
Validazioni: lunghezza minima su username, name, lastName; default e limiti su dateOfBirth; verifica secure su password

Costruttori:

- Password in chiaro → hash + salt generati -> dati serializzati e salvati
- Hash Password e salt noti (per deserializzazione)

Pattern utilizzati:

- Factory per PasswordHasher
- Template Method per build()
- Encapsulation su attributi sensibili



Serializzazione: Estende jsonEntity, salva su file “user/<id>.json”

Note: hashing con PBKDF2WithHmacSHA256, salt generato con SecureRandom. Password codificata Base64, verificabile ma non leggibile

Client

Ruolo: Rappresenta un utente “cliente” (utente comune). Tiene traccia dei ristoranti preferiti dell’utente tramite un set.

Attributi principali: favouriteRestaurants (Set<UUID>)

Relazioni: estende User, aggrega Restaurant tramite id

Validazioni: -

- Controllo per null sulla lista di preferiti passata in costruttore.
- Controllo per null su addFavourite() e removeFavourite()
- Garantisce unicità dei preferiti tramite Set

Costruttori:

- Con password in chiaro -> delega a User
- Con hash, salt e set preferiti (per deserializzazione)

Pattern utilizzati:

- Template Method per build()

Serializzazione: Delega a User

Note: nessuna nota

Client		
Client	(String, String, String, String, String, String, Location, Local	
Client	(String, String, String, String, Location, LocalDate)	
Client	(String, String, String, String, String, String, Location, Local	
removeFavourite	(Restaurant)	boolean
addFavourite	(Restaurant)	boolean
toString	()	String
build	()	void

Owner

Ruolo: Rappresenta un proprietario di uno o più ristoranti. Tiene traccia dei ristoranti pubblicati dall'utente tramite un set.

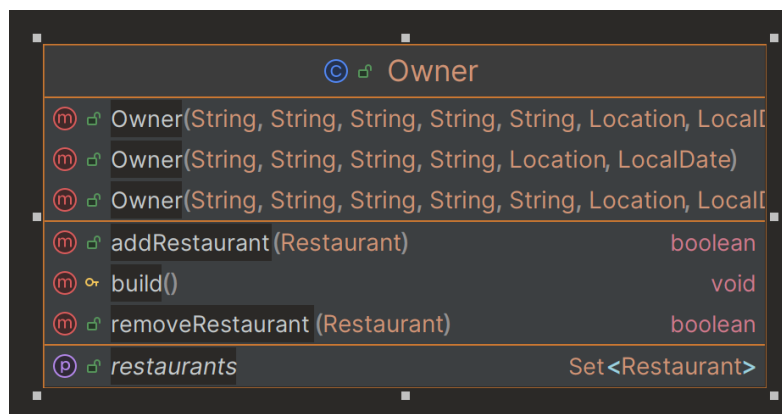
Attributi principali: Set<Restaurant> restaurants (Insieme dei ristoranti pubblicati dall'utente)

Relazioni: Estende User, contiene riferimenti ad oggetti Restaurant su Set.

Validazioni: Controllo per null su metodi di operazioni del set.

Costruttori:

- Con password in chiaro -> delega a User
- Con hash, salt e set ristoranti (per deserializzazione)



Serializzazione: Delega a User

Pattern utilizzati:

- Template Method per build()
- Encapsulation, accesso solo tramite getter, attributi private final

Note:

Implementa il pattern "Template Method" con override del metodo build() nelle sottoclassi.

Location

Ruolo: Modello che rappresenta la posizione geografica di un entità

Attributi principali:

- String address: indirizzo
- Nation country: nazione
- String city: città
- Long latitude: coordinate di latitudine
- Long longitude: coordinata di longitudine

Relazioni: Incluso da Restaurant e User come attributo per indicare la posizione geografica

Validazioni:

- Controlli per null su indirizzo e città
- Controllo di segno per latitudine e longitudine
- Controllo per presenza di nazione in enum apposito.

Serializzazione: Estende JsonEntity, persistente unicamente come attributo di entità

Note:

Implementa il pattern “Template Method” con override del metodo build() nelle sottoclassi.

Location		
Location	(Nation, String, double, double, String)	
toString()		String
build()		void
save()		boolean
city		String
address		String
longitude		double
nation		Nation
latitude		double

Enums (Award, PriceRange, CuisineType, Currency, Nation)

Ruolo: Classi enumerative per valori fissi sicuri (premi, valute, tipi cucina...).

Attributi principali: valori fissi (nazioni, stili di cucina, fasce di prezzo Michelin,...), mappa di valori fissi per lookup con tempo costante

Relazioni: Usate da Restaurant o Location e classi di UI per rappresentare in modo sicuro valori fissi

Validazioni: Controllo per null e richiedono di sanificare l'input per garantire compatibilità

Serializzazione: Persistente unicamente come attributo di entità

Pattern utilizzati: nessuno

Note: nessuna

E ↗ CuisineType (M) ↗ CuisineType() (S) ↗ values() CuisineType [] (M) ↗ toString() String (S) ↗ valueOf(String) CuisineType	E ↗ Award (M) ↗ Award(int) (S) ↗ valueOf(String) Award (S) ↗ fromInt(int) Award (M) ↗ toString() String (S) ↗ values() Award []
E ↗ Nation (M) ↗ Nation(String) (S) ↗ values() Nation [] (M) ↗ toString() String (S) ↗ valueOf(String) Nation (P) ↗ isoCode String	E ↗ PriceRange (M) ↗ PriceRange(String, int, int) (S) ↗ valueOf(String) PriceRange (S) ↗ byDollarAmount(int) PriceRange (S) ↗ values() PriceRange [] (P) ↗ symbol String (P) ↗ maxPrice int (P) ↗ minPrice int

Loader

Ruolo: Lettore di memoria persistente tramite deserializzazione JSON. Riempie Map statiche per accesso efficiente ai dati.

Attributi principali:

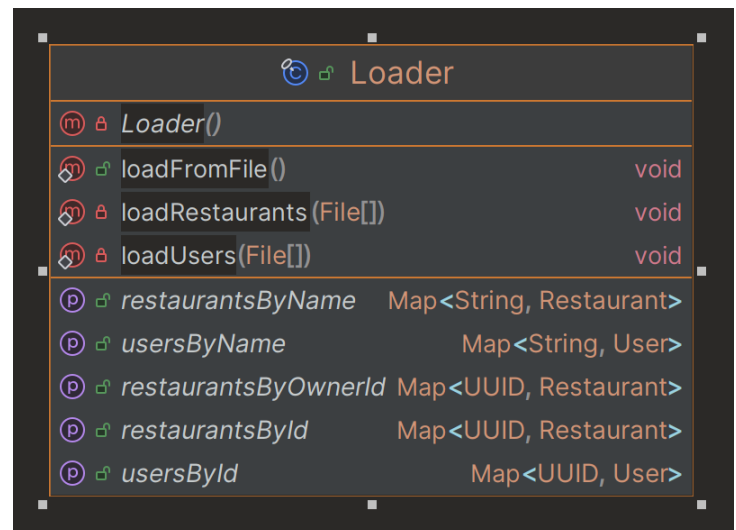
- Map<UUID/String, Restaurant> : Indicizza ristoranti per nome o id per lookup efficiente
- Map<UUID/String, User> : Indicizza utenti per nome o id per lookup efficiente

Relazioni: Utilizzata dalla classe main durante il

Validazioni:

- Controllo di esistenza della directory principale e delle sottodirectory di memoria persistente
- Catch di IOException, SecurityException, IllegalArgumentException
- Valori di fallback sugli attributi
- Gestione Enum con try-catch su CuisineType

Note:



IO

Ruolo: Gestore I/O per menu testuale.

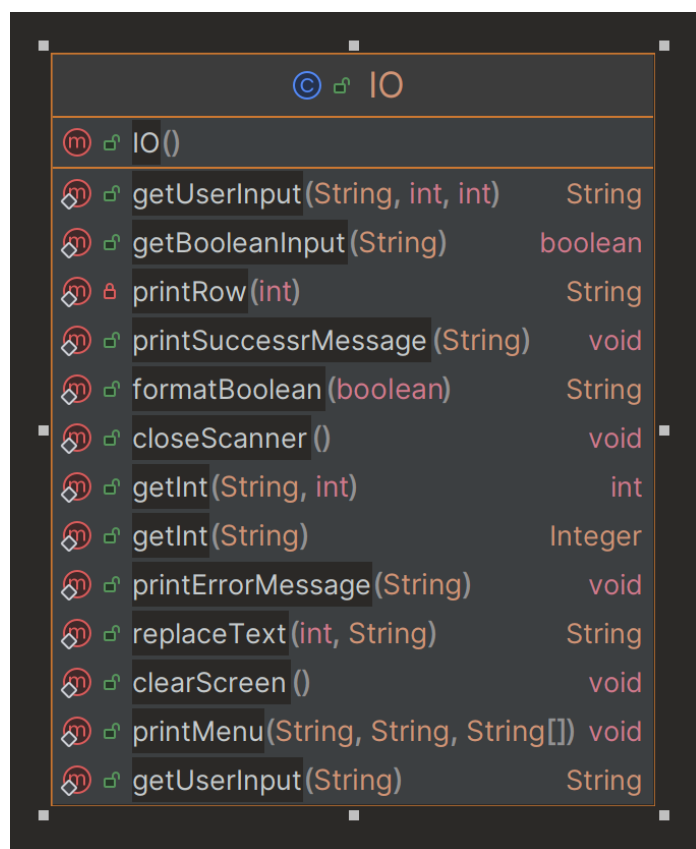
Attributi principali: UUID id: identificatore univoco di oggetto

Relazioni: estesa da User, Restaurant, Review, Locaion

Validazioni: Il metodo save() controlla l'esistenza della cartella di destinazione (saveFolder) e la crea se assente

Note:

Implementa il pattern "Template Method" con



IO	
IO()	
getUserInput(String, int, int)	String
getBooleanInput(String)	boolean
printRow(int)	String
printSuccessrMessage(String)	void
formatBoolean(boolean)	String
closeScanner()	void
getInt(String, int)	int
getInt(String)	Integer
printErrorMessage(String)	void
replaceText(int, String)	String
clearScreen()	void
printMenu(String, String, String[])	void
getUserInput(String)	String

Questa sezione descrive le motivazioni alla base delle scelte algoritmiche adottate nello sviluppo del programma. Le decisioni implementative sono state guidate da criteri specifici legati alla sicurezza, gestione della memoria persistente, validazione robusta degli input e dei dati, impiego di librerie esterne e definizione chiara dei ruoli delle entità.

Ogni scelta è stata orientata a garantire la modularità, semplicità e espandibilità del sistema, con l'obiettivo di mantenere un'architettura e logica gestibile e mantenibile nel tempo, in previsione di evoluzioni o requisiti futuri.

Quanto segue è un riepilogo che presenta le diverse scelte implementative adottate e le motivazioni che ne hanno guidato l'introduzione.

Sicurezza – Gestione della password

Pattern usato: Strategy / Encapsulation

Algoritmo scelto: PBKDF2 con HMAC-SHA256, 10000 iterazioni, salt casuale per ogni utente (16 byte, 256bit key).

Durante la progettazione della classe User, è stata condotta un'analisi comparativa tra diversi algoritmi di hashing delle password, tra cui Argon2, bcrypt, SHA-256, SHA-512, MD5 e PBKDF2.

MD5 è stato escluso in quanto originariamente concepito per la verifica dell'integrità dei file e oggi considerato crittograficamente insicuro per la protezione delle credenziali, a causa delle sue ben note vulnerabilità a collisioni e attacchi di preimage.

La scelta finale è ricaduta su **PBKDF2** (Password-Based Key Derivation Function 2), per la sua capacità di resistere ad attacchi di forza bruta e rainbow table mediante l'impiego congiunto di *salt* e iterazioni.

Questa decisione è stata supportata da fonti tecniche affidabili — in particolare la documentazione delle librerie stesse, tutorial approfonditi come quelli pubblicati da Baeldung e GeeksForGeeks, e discussioni su StackOverflow che hanno fornito utili prospettive pratiche per l'integrazione su contesti più complessi della soluzione.

L'implementazione adottata fa uso esclusivamente delle librerie standard offerte dal JDK, garantendo portabilità e assenza di dipendenze esterne. A causa di instabilità nell'ambiente di sviluppo, è stato scelto di ridurre il numero di iterazioni a 10000.

TheKnife — Scelte Algoritmiche

Quanto segue è uno snippet inerente alla classe interna a User, PasswordHasher, responsabile di gestire l'operazione di Hash:

```
/**
 * Esegue l'hashing della password usando PBKDF2 con il salt e iterazioni.<p>
 *
 * @param password password in char array
 * @param saltBytes salt in byte array
 * @return hash codificato in Base64
 */
private String run(char[] password, byte[] saltBytes) { 3 usages @Luke
    try {
        PBEKeySpec spec = new PBEKeySpec(password, saltBytes, ITERATIONS, KEY_LENGTH);
        SecretKeyFactory skf = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");

        byte[] hash = skf.generateSecret(spec).getEncoded();
        return Base64.getEncoder().encodeToString(hash);
    } catch (Exception e) {
        throw new RuntimeException("Password hashing failed", e);
    }
}
```

Ed i costruttori della stessa, responsabili di preparare il plain text per l'operazione di Hashing:

```
/**
 * Costruttore che genera un nuovo salt casuale e calcola l'hash della password.<p>
 *
 * @param password password in chiaro
 */
public PasswordHasher(String password) { 2 usages @Luke *
    byte[] saltBytes = new byte[SALT_LENGTH];
    new SecureRandom().nextBytes(saltBytes);
    this.salt = Base64.getEncoder().encodeToString(saltBytes);

    char[] plainPassword = password.toCharArray();

    this.pHash = run(plainPassword, saltBytes);
    Arrays.fill(plainPassword, '\0');
}

/**
 * Costruttore che riceve un salt esistente e calcola l'hash della password.<p>
 *
 * @param password password in chiaro
 * @param salt salt esistente in formato stringa
 */
public PasswordHasher(String password, String salt) { 1 usage @Luke
    byte[] saltBytes = salt.getBytes();
    this.salt = Base64.getEncoder().encodeToString(saltBytes);

    char[] plainPassword = password.toCharArray();

    this.pHash = run(plainPassword, saltBytes);
    Arrays.fill(plainPassword, '\0');
}
```

Memoria Persistente – Serializzazione/Deserializzazione JSON

Tecnologia: Jackson ObjectMapper

Pattern: Builder / Factory per deserializzazione controllata

Durante la progettazione del sistema di memoria, è stata posta attenzione al trovare una soluzione efficiente per consentire la permanenza dei dati. Sono state considerate diverse opzioni tra cui strutture relazionali come SQL, nello specifico H2 o SQLite, data la loro indipendenza da strutture esterne come un server e visto anche la dimensione del progetto, a strutture serializzate come CSV e JSON.

Tuttavia, visti i requisiti, si è deciso di implementare la serializzazione tramite JSON, optando per un approccio misto basato sulla libreria Jackson.

La serializzazione è stata implementata tramite un pattern simile al Builder, centralizzato nel metodo `build()` ereditato dalla classe astratta `JsonEntity`. Ogni classe che rappresenta un'entità del dominio, come `User`, `Client`, `Owner` o `Restaurant`,..., sovrascrive questo metodo per popolare manualmente la rappresentazione JSON dell'entità. Questa scelta consente di mantenere pieno controllo sul formato dei dati esportati e sulla coerenza del contenuto generato.

Per quanto riguarda la deserializzazione, si è evitato l'utilizzo diretto della riflessione automatica o delle annotazioni Jackson come `@JsonProperty`, optando invece per un processo esplicito e controllato attraverso la classe `Loader`. Questa classe legge i file JSON tramite `ObjectMapper.readTree()` e costruisce gli oggetti a partire dai nodi JSON rilevanti. Tale approccio, pur richiedendo più codice, garantisce un maggiore controllo, consentendo una validazione precisa dei campi in fase di lettura.

Esempio di costruttore per entità di dominio

Costruttore principale che crea una nuova entità con UUID casuale.
Inizializza la struttura JSON e i riferimenti al file di salvataggio.

Params: `folderName` – nome della cartella di salvataggio relativa a `Constants.ROOT`

```
public JsonEntity(String folderName) { 3 usages  @ Luke
    this.jsonObject = mapper.createObjectNode();
    this.id = UUID.randomUUID();

    jsonObject.put("id", String.valueOf(id));

    this.saveFolder = new File(Constants.ROOT, folderName);
    this.saveFile = new File(saveFolder, "id" + ".json");
}
```

Esempio di costruttore per attributo

Costruttore di default che crea un'entità senza id né file di salvataggio.

```
public JsonEntity() { 3 usages  @ Luke
    this.jsonObject = mapper.createObjectNode();

    this.saveFolder = this.saveFile = null;
    this.id = null;
}
```

Quanto segue è uno snippet inerente alla serializzazione tramite il metodo build()

```
/**
 * Costruisce la rappresentazione JSON dell'utente.<p>
 * Inserisce id, username, nome, cognome, data di nascita e posizione.<p>
 * Inserisce anche la password hashata e il salt.<p>
 */
@Override 20 usages 2 overrides & Luke
protected void build() {
    jsonObject.put( fieldName: "id", String.valueOf(getId()))
        .put( fieldName: "username", username)
        .put( fieldName: "name", name)
        .put( fieldName: "lastName", lastName);

    jsonObject.put( fieldName: "dateOfBirth", dateOfBirth.toString());
    if (location != null) jsonObject.set("location", location.getJsonObject());

    ObjectNode password = mapper.createObjectNode()
        .put( fieldName: "salt", hasher.salt)
        .put( fieldName: "password", hasher.pHash);

    jsonObject.set("password", password);
}
```

Primo esempio utilizzando la superclasse User.

```
/**
 * Ricostruisce l'oggetto JSON del ristorante con tutti i dati aggiornati.<p>
 * Imposta proprietà base, location, tipi di cucina, servizi e recensioni.<p>
 */
@Override 20 usages & Luke
protected void build() {
    jsonObject.put( fieldName: "owner", owner.getId().toString())
        .put( fieldName: "name", name)
        .put( fieldName: "address", description)
        .put( fieldName: "phone", phone)
        .put( fieldName: "priceRange", priceRange.getSymbol())
        .put( fieldName: "award", award.getValue())
        .put( fieldName: "greenStar", greenStar);

    if (loc != null) jsonObject.set("location", loc.jsonObject);

    cuisinesTypesArray.removeAll();
    cuisinesTypes.forEach( CuisineType c -> cuisinesTypesArray.add(c.toString()));
    jsonObject.set("cuisinesTypes", cuisinesTypesArray);

    servicesArray.removeAll();
    services.forEach(servicesArray::add);
    jsonObject.set("services", servicesArray);

    reviewsArray.removeAll();
    reviews.forEach(( UUID u, Review r) -> reviewsArray.add(r.jsonObject));
    jsonObject.set("reviews", reviewsArray);
}
```

Secondo esempio, usando la entità Risorante.

```
/**
 * Salva l'entità su file JSON, creando la cartella se non esiste.<p>
 * Costruisce prima la rappresentazione JSON chiamando {@link #build()}.<p>
 *
 * @return true se il salvataggio è andato a buon fine, false in caso di errore
 */
public boolean save() { 2 overrides  @Luke
    try {
        if (!saveFolder.exists()) saveFolder.mkdirs();
        build();
        mapper.writerWithDefaultPrettyPrinter().writeValue(saveFile, jsonObject);
        return true;

    } catch (IOException | SecurityException e) {
        return false;
    }
}
```

Esempio di salvataggio della entità serializzata su file

Caricamento dati – Inizializzazione centralizzata non lazy

Il caricamento dei dati persistenti dal file system (utenti e ristoranti) è stato centralizzato all'interno della classe Loader, progettata come una classe di utilità statica, tramite la notation `@UtilityClass` di Lombok a responsabilità singola, dotata di metodi statici.

Questo pattern è affine al Singleton, un riferimento di memoria condiviso, convenzionalmente tramite una sistema di cache, riducendo la duplicazione del codice e semplificando la logica di sincronizzazione.

Tuttavia, la mancanza di lazy initialization (cioè il caricamento ritardato al primo accesso) e l'assenza di sincronizzazione (assenza di thread-safety) rendono questo approccio potenzialmente fragile se esteso a contesti multi-threaded, dove più istanze potrebbero accedere contemporaneamente allo stesso stato condiviso.

Questa sezione elenca e discute i limiti del sistema, proponendo anche delle possibili soluzioni per implementazioni future. Poichè, nonostante il sistema segua i principi di separazione delle responsabilità, sia strutturato per essere scalabile, modulare e sicuro, il sistema presenta ancora alcuni limiti strutturali e architetturali, comune alle applicazioni “standalone”, specialmente in ambito didattico, ma che devono essere presi in considerazione in scenari più complessi o per requisiti maggiori:

Classe Loader non implementa logica thread-safe e lazy-loading

Il caricamento dei dati avviene in fase iniziale attraverso una classe Loader. Sebbene efficace in un ambiente monothread, la mancanza di logica concorrente e dell’implementazione del “lazy-loading”, ne limita l'uso in contesti concorrenti o distribuiti.

Un Singleton thread-safe o un meccanismo di lazy loading migliorerebbe l'affidabilità in scenari multiutente.

Jackson: rischio potenziale di deserializzazione non sicura

La libreria Jackson, pur essendo facile da implementare e flessibile, può esporre a vulnerabilità note, come exploit tramite notation @class o oggetti JSON malevoli.

Tali rischi sono mitigati evitando la deserializzazione automatica e ricostruendo manualmente ogni oggetto tramite JsonNode, applicando filtri e validazioni esplicite.

Persistenza semplificata su file system

Il salvataggio e caricamento dei dati avviene tramite file JSON. Pur essendo leggibile e adatto a scopi didattici, tale approccio non supporta atomicità, rollback o query complesse (come ad esempio subqueries).

In un contesto reale, l’uso di un database relazionale o documentale sarebbe necessario per garantire scalabilità, sicurezza e coerenza.

Strutture dati non concorrenti

Le collezioni principali (Map, Set) non sono sincronizzate, rendendo l'intera architettura inadatta a un utilizzo concorrente senza ulteriori meccanismi di protezione

🔗 Jackson JSON Processor

<https://github.com/FasterXML/jackson>

Documentazione ufficiale e best practice per la serializzazione/deserializzazione.

🔗 PBKDF2 Password Hashing

- Baeldung – Secure Password Storage:
 - <https://www.baeldung.com/java-password-hashing>
- GeeksForGeeks – Hashing in Java:
 - <https://www.geeksforgeeks.org/java/hashing-in-java/>
- Coding Tech Room – Securely Storing Passwords in Java: Best Practices and Techniques
 - <https://codingtechroom.com/tutorial/java-java-storing-passwords>
- Java Cryptography Architecture (JCA) Documentation
 - <https://docs.oracle.com/en/java/javase/11/security/java-cryptography-architecture-jca-reference-guide.html>

🔗 Hashing e sicurezza

- NIST Recommendations for Password Storage
 - <https://csrc.nist.gov/pubs/sp/800/132/final>
- Vari thread su StackOverflow riguardo gestione sicura di salting, hash e crittografia. In particolare:
 - <https://stackoverflow.com/questions/2860943/how-can-i-hash-a-password-in-java>
 - <https://stackoverflow.com/questions/30496061/why-not-use-md5-for-password-hashing>
 - <https://stackoverflow.com/questions/74279342/verifying-hash-password-pbkdf2-with-java>



TheKnife – Javadoc

Disponibile a questo indirizzo:

<https://the-knife-project.vercel.app/>